MODULE 3 · LECTURE 15

CS 6530 Applied Cryptography

Elliptic-Curve Foundations

Module 3 – Public-Key Cryptography

MOTIVATION

Why Elliptic Curve Cryptography?

Public-key cryptography needs hard mathematical problems

RSA

Large-integer arithmetic

Finite-Field Diffie–Hellman

Modular arithmetic

↓

As security requirements increase, can we retain strong security with much smaller parameters?

↓

ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

Elliptic curves provide groups for public-key cryptography using much smaller parameters.

To reach ≈128-bit classical security, compare parameter sizes:

RSA — **3072-bit modulus**

versus

ECC — **≈256-bit parameters**

ECC → strong classical public-key security with much smaller parameters.

Elliptic Curves: Start with the Geometry

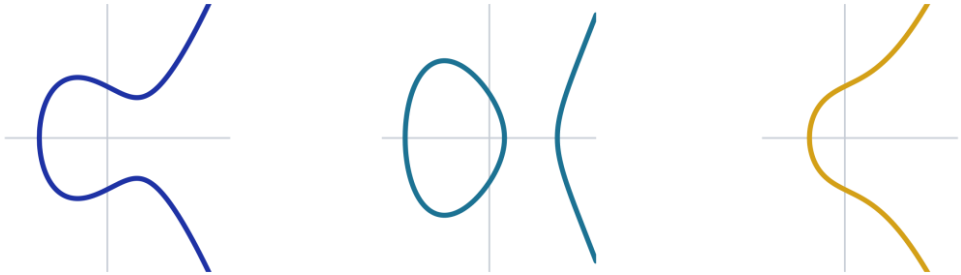
E: $y^2 = x^3 + ax + b$

$a, b, x, y \in \mathbb{R}$

$4a^3 + 27b^2 \neq 0$

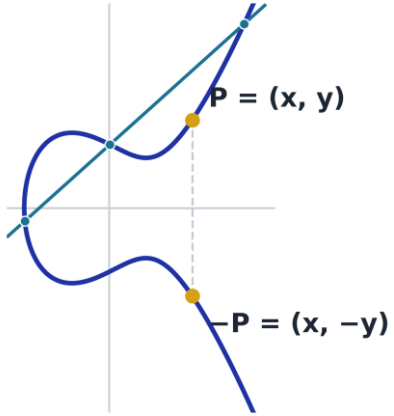
Non-singular: no cusps or self-intersections
Despite the name, an elliptic curve is not an ellipse.

Different (a, b) → different elliptic curves over $\mathbb{R}$



(a, b) = (−1, 1) (a, b) = (−3, 1) (a, b) = (1, 1)

a and b define the curve; over $\mathbb{R}$, changing them reshapes it.



- **Symmetric about the x-axis**
  $(x, y) \in E \Rightarrow (x, -y) \in E$

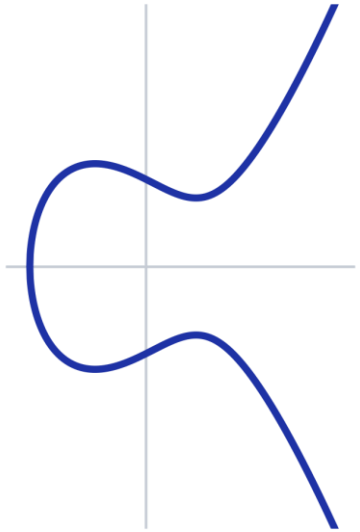
- A non-vertical line intersects the cubic in at most three points.

a and b select the curve; over the reals, we can visualize its geometry.

Cryptography Moves the Curve to a Finite Field

Over the real numbers

$E(\mathbb{R})$: $y^2 = x^3 + ax + b$



Continuous curve

Useful for geometric intuition

SAME EQUATION

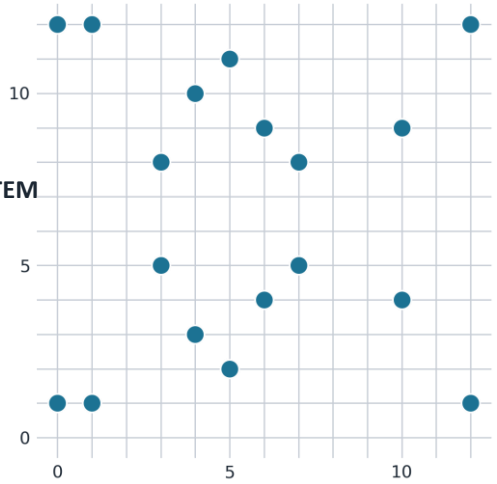
DIFFERENT NUMBER SYSTEM

$\mathbb{R} \rightarrow \mathbb{F}_p$

Over a prime finite field

$E(\mathbb{F}_p)$: $y^2 \equiv x^3 + ax + b \pmod{p}$

$x, y, a, b \in \mathbb{F}_p$



Finite set of discrete points

$\mathbb{F}_p = \{0, 1, \ldots, p-1\}$

Arithmetic is performed modulo p.

Choose a prime $p > 3$.

$4a^3 + 27b^2 \not\equiv 0 \pmod{p}$

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 \equiv x^3 + ax + b \pmod{p}\} \cup \{O\}$$

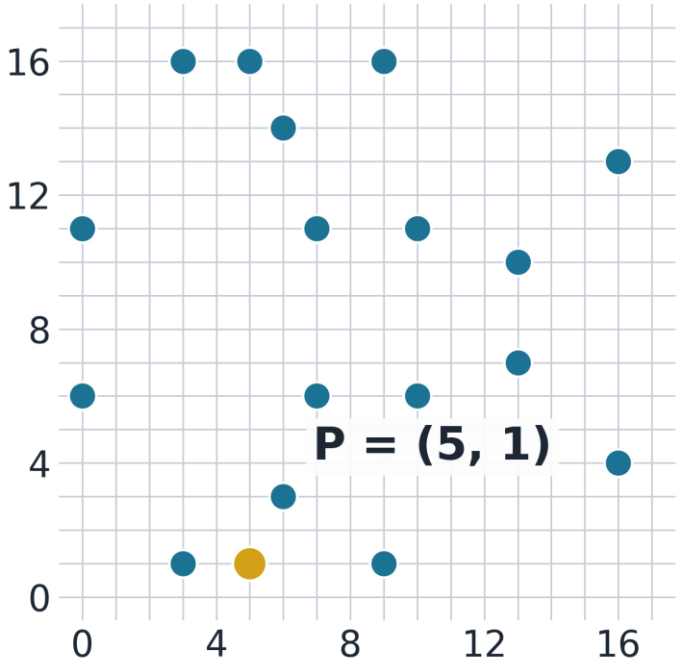
O — a special point at infinity; we will shortly see why it is needed.

The real curve gives geometric intuition; cryptographic ECC performs arithmetic on discrete points over a finite field.

Our Running Curve over $\mathbb{F}_{17}$

E: $y^2 \equiv x^3 + 2x + 2 \pmod{17}$

$p = 17 \quad a = 2 \quad b = 2$



Every point actually satisfying E

$\mathbb{F}_{17} = \{0, 1, \ldots, 16\}$, with arithmetic modulo 17

Verify P = (5, 1)

$y^2 = 1^2 = 1$

$x^3 + 2x + 2 = 5^3 + 2(5) + 2 = 137$

$137 \equiv 1 \pmod{17}$

Therefore $1 \equiv 1 \pmod{17}$ ✓

$P = (5, 1) \in E(\mathbb{F}_{17})$

What about (5, 2)?

$y^2 = 2^2 = 4$, but $5^3 + 2(5) + 2 \equiv 1 \pmod{17}$

$(5, 2) \notin E(\mathbb{F}_{17})$

$4a^3 + 27b^2 = 4(2^3) + 27(2^2) = 140$

$140 \equiv 4 \pmod{17} \neq 0 \rightarrow$ curve is non-singular

A point belongs to $E(\mathbb{F}_p)$ only if its coordinates satisfy the curve equation modulo p.

THE ALGEBRAIC STRUCTURE

Elliptic-Curve Points Form a Group

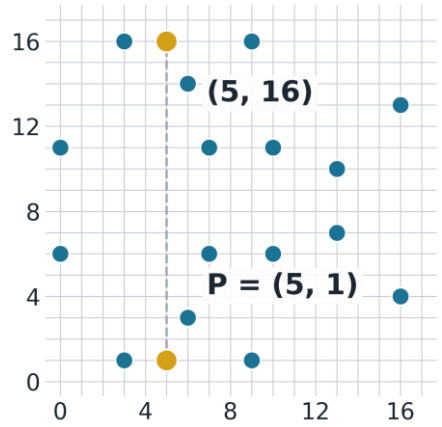
$E(\mathbb{F}_{17})$ = {valid affine points} ∪ {O}

1. CLOSURE

$P, Q \in E \;\Rightarrow\; P + Q \in E$

2. IDENTITY

O = point at infinity; $P + O = O + P = P$



P and −P: vertically aligned

3. INVERSE

For $P = (x, y)$: $-P = (x, -y \bmod p)$

$P + (-P) = O$

Example: $P = (5, 1) \;\Rightarrow\; -P = (5, 16)$

(5, 1) + (5, 16) = O

4. ASSOCIATIVITY

$(P + Q) + R = P + (Q + R)$

5. COMMUTATIVITY

$P + Q = Q + P$

$E(\mathbb{F}_p)$, under point addition, is an ABELIAN GROUP.

Finite-Field DH: $(\mathbb{F}_p^*, \times)$ vs ECC: $(E(\mathbb{F}_p), +)$ — ECC changes the group; the group-based idea remains.

TERMS — Affine point: ordinary curve point (x, y). **Abelian group:** a group whose operation is commutative.

ECC uses a group whose elements are curve points and whose operation is point addition.

DEFINING THE GROUP OPERATION

How Do We Add Elliptic-Curve Points?



$P + Q = ?$

If "+" is our group operation, what does adding two curve points mean?

CASE 1 — Identity

$P + O = P$

CASE 2 — Inverse / vertical case

$Q = -P \rightarrow P + (-P) = O$

$P \neq Q$ and $Q \neq -P$

Define $R = P + Q$

POINT ADDITION

$P = Q$

$P + P = 2P$

POINT DOUBLING

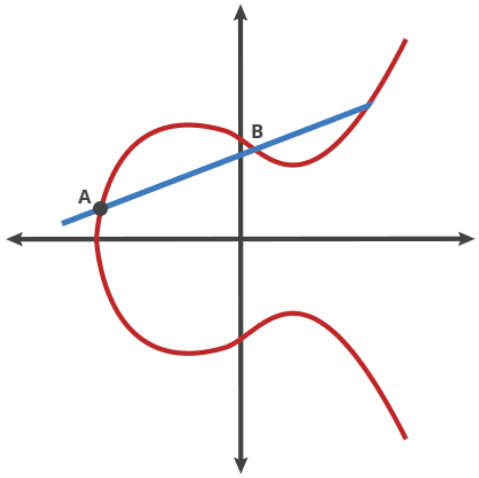
Edge case: if $y = 0$, then $P = -P$, so $2P = O$

How can two points P and Q produce another point $P + Q$ that is still on the curve?

NEXT → The geometry gives us the rule.

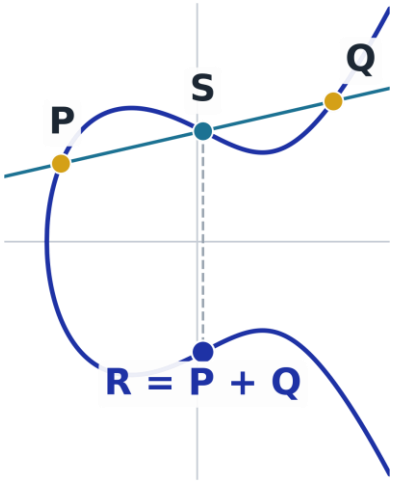
Point addition must handle ordinary points, doubling, the identity O, and inverse points.

Point Addition: The Geometry Gives the Rule



For geometric intuition, work over $E(\mathbb{R})$. Let P and Q be two distinct points:

$P \neq Q \qquad Q \neq -P$



1 — JOIN: Draw the line through P and Q.

2 — INTERSECT: The line meets the curve at a third point S.

3 — REFLECT: Reflect S across the x-axis to obtain R.

$$P + Q = R = -S$$

where S = third intersection, R = reflected point = $P + Q$

Also: $P + Q + S = O$

? THINK LIKE AN ENGINEER

Why is the third intersection S reflected before we call the result P + Q?

INSIGHT

Chord-and-reflect defines the addition rule:

$P + Q = -S$

FINITE-FIELD BRIDGE

Fp has no continuous curve; the same group law uses finite-field arithmetic.

Join P and Q → third intersection → reflect → $P + Q$.

Point Addition over $\mathbb{F}_p$

For $P = (x_1, y_1)$, $Q = (x_2, y_2)$ with $P \neq Q$, $Q \neq -P$, let $R = P + Q = (x_3, y_3)$

GENERAL RULE

$\lambda = (y_2 - y_1)(x_2 - x_1)^{-1} \bmod p$

$x_3 = \lambda^2 - x_1 - x_2 \bmod p$

$y_3 = \lambda(x_1 - x_3) - y_1 \bmod p$

WORKED EXAMPLE

E: $y^2 \equiv x^3 + 2x + 2 \pmod{17}$ **$P = (5,1)$ $Q = (9,16)$**

$\lambda = (16 - 1)(9 - 5)^{-1} \bmod 17 = 15(4^{-1}) \bmod 17$

Since $4^{-1} \equiv 13 \pmod{17}$ since $4 \times 13 = 52 \equiv 1 \pmod{17}$

$\lambda = 15 \times 13 = 195 \equiv 8 \pmod{17}$

$x_3 = 8^2 - 5 - 9 = 50 \equiv 16 \pmod{17}$

$y_3 = 8(5 - 16) - 1 = -89 \equiv 13 \pmod{17}$

(5, 1) + (9, 16) = (16, 13)

? THINK LIKE AN ENGINEER

There is no ordinary division modulo p.

What does division by (x2 − x1)

require?

INSIGHT

Multiplication by the modular inverse:

$1 / (x_2 - x_1)$ → **$(x_2 - x_1)^{-1} \bmod p$**

Finite-field point addition implements the group law using modular arithmetic and modular inverses.

Point Doubling: P + P = 2P

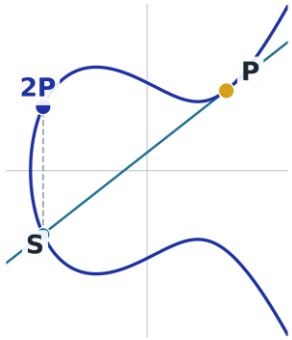
As Q approaches P ($Q \to P$), the chord through P and Q becomes the tangent at P.

FINITE-FIELD RULE

For $E: y^2 = x^3 + ax + b$, $P = (x_1, y_1)$ with $y_1 \neq 0$:

$\lambda = (3x_1^2 + a)(2y_1)^{-1} \bmod p$ $\quad x_3 = \lambda^2 - 2x_1 \bmod p$

$y_3 = \lambda(x_1 - x_3) - y_1 \bmod p$



Tangent at P → 2nd point S → reflect → 2P

WORKED EXAMPLE

E: $y^2 \equiv x^3 + 2x + 2 \pmod{17}$ **$P = (5, 1)$**

$\lambda = [3(5^2) + 2](2 \times 1)^{-1} \bmod 17$

$3(25) + 2 = 77 \equiv 9 \pmod{17}$

$2^{-1} \equiv 9 \pmod{17}$ since $2 \times 9 = 18 \equiv 1 \pmod{17}$

$\lambda = 9 \times 9 = 81 \equiv 13 \pmod{17}$

$x_3 = 13^2 - 2(5) = 159 \equiv 6 \pmod{17}$

$y_3 = 13(5 - 6) - 1 = -14 \equiv 3 \pmod{17}$

$2P = (6, 3)$

? THINK LIKE AN ENGINEER

Why can't we use the addition slope formula when P = Q?

INSIGHT

P = Q gives x2 − x1 = 0, which has no inverse.

EDGE CASE

If y1 = 0, the tangent is vertical:

$2P = O$

Point doubling replaces the chord by a tangent and produces 2P.

Your Turn: Compute 3P

Use: $E: y^2 \equiv x^3 + 2x + 2 \pmod{17}$ **$P = (5, 1)$** **$2P = (6, 3)$**

YOUR TASK: Compute $3P = 2P + P = ?$

POINT-ADDITION RULE

$\lambda = (y_2 - y_1)(x_2 - x_1)^{-1} \bmod p \qquad x_3 = \lambda^2 - x_1 - x_2 \bmod p$

$y_3 = \lambda(x_1 - x_3) - y_1 \bmod p$

PAUSE AND TRY

SOLUTION

Using $P = (5, 1)$, $2P = (6, 3)$:

$\lambda = (3 - 1)(6 - 5)^{-1} \bmod 17 = 2 \times 1^{-1} = 2$

$x_3 = 2^2 - 5 - 6 = -7 \equiv 10 \pmod{17}$

$y_3 = 2(5 - 10) - 1 = -11 \equiv 6 \pmod{17}$

$3P = (10, 6)$

? THINK LIKE AN ENGINEER

We obtained 2P, then 3P = 2P + P. To compute kP, how many point additions would I need?

Repeated point addition builds: P, 2P, 3P, … How does this scale for a large k?

Scalar Multiplication: Building P, 2P, 3P, ...

For integer $k > 0$: $\boldsymbol{kP = P + P + \ldots + P}$ *(k copies)* — **SCALAR MULTIPLICATION**

IMPORTANT: *kP is NOT coordinate scaling — it's repeated group addition.*

Example: $E: y^2 \equiv x^3 + 2x + 2 \pmod{17}$, $\boldsymbol{P = (5, 1)}$

k	kP
1	(5,1)
2	(6,3)
3	(10,6)
4	(3,1)
5	(9,16)
6	(16,13)
7	(0,6)
8	(13,7)
9	(7,6)
10	(7,11)

k	kP
11	(13,10)
12	(0,11)
13	(16,4)
14	(9,1)
15	(3,16)
16	(10,11)
17	(6,14)
18	**(5,16)**
19	O

$$\boldsymbol{P \rightarrow 2P \rightarrow 3P \rightarrow \ldots \rightarrow 18P \rightarrow O}$$

Notice the inverse pattern: $P = (5,1)$, $18P = (5,16) = -P \Rightarrow 19P = P + (-P) = O$

GROUP ORDER: $\boldsymbol{\#E(\mathbb{F}_{17}) = 19}$ **POINT ORDER:** $\boldsymbol{\text{ord}(P) = 19}$ *(smallest k with $kP = O$)*

They are equal here — P generates the entire group. ($20P = P$, $21P = 2P$, ...)

? THINK LIKE AN ENGINEER

The curve has finitely many points — what happens to P, 2P, 3P, ...?

INSIGHT

The sequence reaches O and repeats: $\boldsymbol{19P = O}$

$\#E(F_p)$ gives the group size; ord(P) is when P's multiples first reach O. Here both are 19, so P generates the group.

Computing kP Efficiently: Double-and-Add

Computing kP literally needs $k - 1$ additions — impractical for cryptographic-size k.

KEY IDEA

Use the binary form of k: $13 = (1101)_2 = 8 + 4 + 1$

$$13P = 8P + 4P + P$$

BUILD POWERS OF TWO BY DOUBLING

$P \rightarrow 2P \rightarrow 4P \rightarrow 8P$ *(each step: double)*

FOLLOW THE BITS: $13 = (1101)_2$

BIT	ACCUMULATOR R
1	P
1	3P
0	6P
1	13P

DOUBLE-AND-ADD ALGORITHM

Start: R ← O
For each bit of k (MSB → LSB):
  R ← 2R
  If the bit is 1: R ← R + P
Return: R = kP

FOR OUR RUNNING EXAMPLE

$P = (5,1)$ $4P = (3,1)$ $8P = (13,7)$
$8P + 4P = 12P = (0,11)$
$12P + P = 13P = (16,4)$

13P = (16, 4)

EFFICIENCY: *naive ~ k ops; double-and-add ~ $\log_2 k$ — a 256-bit scalar needs only hundreds of EC*

? THINK LIKE AN ENGINEER

Why does representing k in binary make scalar multiplication practical?

INSIGHT

Binary decomposition reuses doubling; adds only where bits require it.

IMPLEMENTATION NOTE: *production systems harden this against side-channel leakage.*

Double-and-add computes kP efficiently from the bits of k, using repeated doubling and selected point additions.

The Elliptic-Curve Discrete Logarithm Problem

SCALAR MULTIPLICATION — EASY

Given P and k, compute $Q = kP$

P, k —efficient→ $Q = kP$

REVERSE: ECDLP — HARD

Given P and $Q = kP$, determine k = ?

P, Q —ECDLP→ k = ?

ELLIPTIC-CURVE DISCRETE LOGARITHM PROBLEM (ECDLP)

TOY EXAMPLE

$E: y^2 \equiv x^3 + 2x + 2 \pmod{17}$, $P = (5,1)$

Suppose $Q = (16,4)$. Enumerate $P, 2P, 3P, \ldots$ and find: $13P = (16,4) \rightarrow k = 13$

WHY ISN'T THIS SECURE?

Our teaching example has ord(P) = 19 — only a tiny number of possibilities need testing. Real cryptographic groups use an enormously larger prime-order subgroup.

GENERIC ATTACK COMPLEXITY

For a subgroup of large prime order n, generic ECDLP attacks (e.g. Pollard's rho) need ~$O(\sqrt{n})$ group operations.

If $n \approx 2^{256}$, then $\sqrt{n} \approx 2^{128}$ — about 128-bit classical security for a properly chosen ~256-bit EC subgroup.

? THINK LIKE AN ENGINEER

Our toy curve has only 19 multiples of P. Why can't an attacker “try every k” on a real curve?

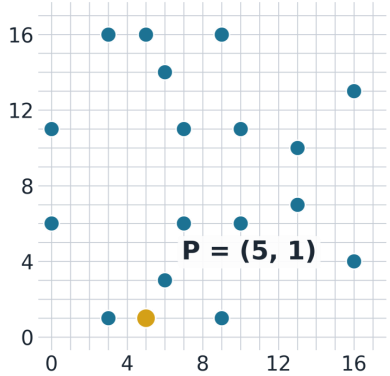
INSIGHT

At cryptographic scale the search space is enormous — even square-root attacks are infeasible.

Computing Q = kP is efficient. Recovering k from P and Q is the ECDLP — computationally hard for properly chosen cryptographic curves.

From Our Toy Curve to Real ECC

OUR TEACHING CURVE



Small enough to plot, calculate, and enumerate.

SAME EC GROUP OPERATIONS

Point addition

↓

Point doubling

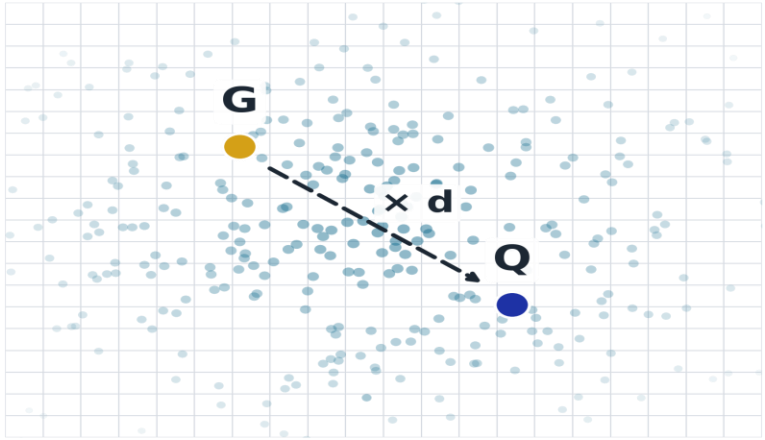
↓

Scalar multiplication

NIST P-256

also known as secp256r1

$\mathbb{F}_p$ *($p \approx$ 256-bit prime), base point G, order n*



Conceptual only — not an actual P-256 point plot.

TOY SCALE → **CRYPTOGRAPHIC SCALE**

≈128-BIT CLASSICAL SECURITY

RSA ≈ 3072-bit modulus *vs* **ECC** ≈ 256-bit parameters

Compact EC parameters reduce communication/storage overhead and suit constrained systems.

Performance depends on implementation; none of these systems are quantum-resistant.

? THINK LIKE AN ENGINEER

If two systems target similar security, why might smaller parameters matter?

INSIGHT

Communication • storage • memory • constrained devices

Same elliptic-curve group mathematics — now at cryptographic scale: efficient scalar multiplication, hard ECDLP, and much smaller parameters than RSA for comparable classical security.

References

PRIMARY TEXT

Christof Paar and Jan Pelzl

Understanding Cryptography: A Textbook for Students and Practitioners
Springer

Chapter 9 — Elliptic Curve Cryptography
§9.1 How to Compute with Elliptic Curves
§9.2 The Elliptic Curve Discrete Logarithm Problem

COURSE / REFERENCE TEXT

Jean-Philippe Aumasson

Serious Cryptography: A Practical Introduction to Modern Encryption
No Starch Press

Chapter 12 — Elliptic Curves

SECURITY-STRENGTH GUIDANCE

NIST SP 800-57 Part 1 Rev. 5

Recommendation for Key Management: Part 1 — General
National Institute of Standards and Technology

Security-strength guidance and comparable cryptographic key sizes.

STANDARDIZED CURVE REFERENCE

NIST FIPS 186-5

Digital Signature Standard (DSS)

National Institute of Standards and Technology

Reference for approved elliptic curves, including NIST P-256 domain parameters.